Skills

A skill is a reusable instruction pack. It is a directory containing a `SKILL.md` file: YAML frontmatter that configures the skill, and a Markdown body that is the instruction text injected into the conversation when the skill runs. Everything else in the directory - scripts, templates, examples, reference material - exists only so the instructions can point at it.

Use a skill when you find yourself pasting the same paragraph of instructions into the chat, or when a task needs a checklist plus a couple of helper scripts that should live with it in version control.

- How the skills system works
- Creating a skill
- Frontmatter reference
- Writing the body
- Invoking and managing skills
- Worked example
- Troubleshooting

How the skills system works

Discovery

At session start ragent scans a fixed, ordered list of directories. Every immediate subdirectory that contains a `SKILL.md` is one skill; the subdirectory name is the skill's default name. Subdirectories without a `SKILL.md` are skipped silently, and a pack whose frontmatter does not parse is logged as a warning and skipped - one broken pack never blocks the others.

The scan order is also the precedence order, lowest priority first:

#	Scope	Directory	Notes
1	`bundled`	compiled into the binary	`simplify`, `batch`, `debug`, `loop`
2	`openskills-global`	`~/.agents/skills/`, `~/.agent/skills/`, `~/.claude/skills/`	AgentSkills convention and its OpenSkills siblings
3	`personal`	`~/.config/ragent/skills/`	shared by every project for this user
4	`personal`	each `skill_dirs` entry from `ragent.json`	a configured extra root
5	`openskills-project`	`.agents/skills/`, `.agent/skills/`, `.claude/skills/` under the working directory	
6	`project`	`.ragent/skills/` under the working directory	
7	`project`	`<subdir>/.ragent/skills/` for each first-level subdirectory	monorepo support

Skill directories contributed by **enabled plugins** are appended to the configured `skill_dirs` list, so a plugin's packs are discovered by the same scan. See plugins.md for the plugin side.

Name clashes

On a clash a higher-numbered scope wins outright. Within one scope the first pack registered wins, which is why the order above matters: a project pack `deploy` overrides a personal `deploy`, which overrides a bundled `deploy`, and a ragent-native directory wins over an AgentSkills directory at the same level. The `scope` column in `/skill` shows which one you actually got.

Lazy loading

Discovery reads only the frontmatter. The Markdown body stays on disk until the skill is invoked, then it is cached in memory for the rest of the session. `/reload skills` (or the `Skill` tool's `reload` action) drops the cached bodies and re-scans, so an edited `SKILL.md` takes effect without a restart.

What happens at invocation

Invoking a skill runs four steps in order:

1. **Body load** - the Markdown body is read from disk (or taken from cache).
2. **Argument substitution** - placeholders in the body are replaced (below).
3. **Dynamic context injection** - !`command` placeholders are executed, but only if the skill opted in with `allow-dynamic-context: true`.
4. **Delivery** - the processed body is wrapped as `[Skill: /<name>]` plus the text and handed to the agent, either in the current conversation or in a forked sub-session when `context: fork` is set.

Creating a skill

1. Create the directory

Put the pack in the scope you want. For a pack that only makes sense in this repository use the project scope:

```
mkdir -p .ragent/skills/deploy
```

Use `~/.config/ragent/skills/deploy` for a pack you want in every project.

2. Write `SKILL.md`

The file **must** start with a `---` line, contain valid YAML, and close with a second `---` line. Everything after the closing delimiter is the body.

```
---
name: deploy
description: Deploy the application to production and verify the rollout
argument-hint: "[environment]"
allowed-tools:
  - Bash
  - Read
---

Deploy `$ARGUMENTS` to production.

1. Run the test suite: `cargo nextest run`
```

```
2. Build the release binary: `cargo build --release`
3. Push and watch the rollout.
```

Minimum viable pack - only `description` is genuinely worth writing, and the directory name supplies the name:

```
---
description: Summarise the open pull requests for this repo
---

Fetch the open PRs and produce a table of title, author, age, and CI status.
```

3. Add supporting files (optional)

Anything else in the directory is yours to use. Conventional subdirectories:

```
.ragent/skills/deploy/
  SKILL.md          # required
  scripts/          # helper scripts the instructions call
  templates/        # files the agent fills in
  examples/         # example output showing the expected format
  resources/        # reference material
```

Refer to them from the body with the `${RAGENT_SKILL_DIR}` placeholder so the instructions work wherever the pack is installed.

4. Verify it loads

```
/skill            # list every registered skill with scope and access
/skill deploy     # invoke it
```

A pack that does not appear in `/skill` failed to parse; the reason is in the session log. Run `/reload skills` after fixing it.

Frontmatter reference

Every key is optional. Keys are kebab-case, exactly as spelled below. An unrecognised key is ignored rather than rejected, so a `SKILL.md` written for another agent tool will still load.

Key	Type	Default	Meaning
`name`	string	directory name	Skill identifier. Must be lowercase letters, digits and hyphens, max 64 chars.
`description`	string	none	What the skill does. Shown in `/skill` and used for agent auto-invocation matching.
`argument-hint`	string	none	Autocomplete hint, e.g. `"[environment]"` or `"[interval] <prompt>"`.
`allowed-tools`	string or list	none (no restriction)	Restricts the tool surface for the run.

Key	Type	Default	Meaning
`user-invocable`	bool	`true`	`false` hides the skill from the `/` menu and from `/name`.
`disable-model-invocation`	bool	`false`	`true` stops the agent invoking the skill itself.
`model`	string	none	`provider/model` or `provider:model` override while the skill is active.
`context`	string	none	`fork` runs the skill in an isolated sub-session.
`agent`	string	`general` when forked	Agent profile for the forked run, e.g. `explore`.
`allow-dynamic-context`	bool	`false`	Opt in to `` !`command` `` execution in the body.
`trigger`	string	the skill name	Trigger phrase recorded in the skill catalog.
`hooks`	mapping	none	Skill-scoped hook definitions, parsed and stored as JSON.
`license`	string	none	Agent Skills specification field, informational.
`compatibility`	string	none	Agent Skills specification field, informational.
`metadata`	mapping	`{}`	Arbitrary string key-value pairs, informational.

Notes on the ones with behaviour behind them:

`name` is validated on load. An empty name, a name over 64 characters, or one containing uppercase letters, spaces, underscores or punctuation is a parse error and the pack is skipped. Omit `name` and the directory name is used, which is why `deploy/SKILL.md` does not need to repeat itself - and why naming a directory `Deploy` is a mistake.

`allowed-tools` accepts a single string (`allowed-tools: Bash`) or a list (`allowed-tools: [Bash, Read]`). Entries resolve against the tool registry *ignoring case*, so `bash`, `Bash` and `BASH` all name the tool spelled `Bash`. The pseudo-entry `any` (any case) lifts the restriction entirely. An entry that matches no registered tool is reported once per skill and entry, naming the skill and the offending entry - a typo does not silently remove a capability without telling you. See tools for the registry names.

`user-invocable` and `disable-model-invocation` are independent. Together they give four states, which `/skill` prints in its `access` column:

`user-invocable`	`disable-model-invocation`	Access	Use when
`true`	`false`	both	The default - anyone can run it, the agent can pick it
`true`	`true`	user-only	A destructive action only a human should start

user-invocable	disable-model-invocation	Access	Use when
false	false	agent-only	Context the agent pulls in itself; nothing to type
false	true	disabled	Parked without deleting it

model is applied on top of the session's active model, so it wins over both the session model and the agent profile's default. A profile that explicitly pinned its model still loses to the skill's **model**. Omit it to inherit.

context: fork creates a new session, runs the body through the agent loop there, and returns only the final response text to the caller - a large investigation does not fill the parent conversation with tool output. Use it for work that is long, exploratory, or expected to fail. Without it, the skill content is injected into the current conversation and the main agent carries on with full history.

allow-dynamic-context is off by default. A body containing **!`ls src/`** is sent to the agent with the placeholder intact unless the skill opts in. This is deliberate: **SKILL.md** files are fetched from marketplaces and cloned from git, so running their commands is an explicit decision.

Writing the body

Argument placeholders

Substituted in this order every time the skill is invoked:

Placeholder	Replaced with	Example
${RAGENT_SESSION_ID}	the current session id	sess-01H...
${RAGENT_SKILL_DIR}	absolute path to the directory holding SKILL.md	/repo/.ragent/skills/deploy
$ARGUMENTS[N]	argument N, 0-based	$ARGUMENTS[0]
$ARGUMENTS	the whole argument string, verbatim	/deploy staging prod -> staging prod
$N	shorthand for $ARGUMENTS[N]	$0, $1

Arguments are split on whitespace, with single and double quotes honouring embedded spaces, so "my report" final yields two arguments. An out-of-range index becomes an empty string - it does not fail.

Prefer $ARGUMENTS when the body treats the input as free text, and the indexed forms when you need to tell argument 1 from argument 2. Document the expected shape in argument-hint.

Dynamic context injection

With allow-dynamic-context: true, backtick-command placeholders are executed before the body is sent:

```
## Pull request context
- Diff: !`gh pr diff`
- Changed files: !`git diff --name-only`
```

Security constraints, all enforced:

- Commands run **directly**, never through `sh -c`, so the body cannot chain a second command or trigger shell expansion.
- Any command containing a shell metacharacter (`|`, `&`, `;`, `<`, `>`, `$`, backtick, `(`, `)`, `{`, `}`, `*`, `?`, `~`, `[`, `]`, `!`, newline) is rejected with an error message in place of the output.
- Only allowlisted executables may run: version control inspection (`git`, `svn`, `hg`), read-only file tools (`cat`, `head`, `tail`, `wc`, `ls`, `find`, `tree`, `file`, `stat`, `du`, `df`, `readlink`, `realpath`, `basename`, `dirname`), text processing (`grep`, `rg`, `cut`, `sort`, `uniq`, `tr`, `diff`, `comm`, `paste`, `column`, `fmt`, `fold`, `jq`, `yq`), and shell info utilities (`echo`, `printf`, `date`, `whoami`, `hostname`, `uname`, `id`, `pwd`, `which`, `test`, `true`, `false`).
- Network clients (`curl`, `wget`, `gh`), interpreters (`python`, `node`, `awk`, `sed`) and build tools that execute arbitrary code (`cargo`, `make`, `npm`) are **not** allowlisted. A command that needs them belongs in a `scripts/` file the agent runs through `Bash` with its normal permission checks, not in the body.
- Each command has a 30-second timeout. A failed or timed-out command puts its error message where the output would have gone, so the agent sees the failure rather than an empty line.

Invoking and managing skills

TUI commands:

Command	Effect
`/skill` or `/skills`	List every registered skill: name, `argument-hint`, scope, access, description
`/skill <name> [args]`	Invoke a pack by name
`/<name> [args]`	The bare trigger form - same as `/skill <name>`
`/skill help`	Show the `/skill` help block
`/reload skills`	Drop cached bodies and re-scan every skill directory

The `Skill` tool exposes the same surface to the agent itself: `list` (with an optional `scope` filter and `include_bodies`), `read` (process a skill's body with optional `arguments` without invoking it), `load` (discover and invoke), and `reload`.

Worked example

`.ragent/skills/release-notes/SKILL.md`:

```
---
name: release-notes
description: Draft release notes for the version in Cargo.toml from the commits since the last to
argument-hint: "[previous-tag]"
allowed-tools:
  - Bash
  - Read
  - Write
disable-model-invocation: true
allow-dynamic-context: true
---
```

```
Draft release notes for `$ARGUMENTS` (default: the most recent tag).

## Current version

!`grep -m1 '^version' Cargo.toml`

## Commits since the previous tag

!`git log --oneline -20`

## Output

Write `docs/releases/notes.md` with a Keep-a-Changelog style section for the
current version: Added, Changed, Fixed, Removed. Omit empty sections. One bullet
per user-visible change; skip pure refactors and CI tweaks.
```

Reading it back:

- `disable-model-invocation: true` - a human decides when notes are drafted.
- `allow-dynamic-context: true` - required for the two `` !`...` `` blocks; both commands (`grep`, `git`) are allowlisted and neither contains a metacharacter.
- `$ARGUMENTS` - passing `/release-notes v1.0.159` names the previous tag; omitting it leaves the body with an empty string and the "default" clause applies.
- `allowed-tools` - the run cannot call anything outside those three tools, which is the whole point of a notes-drafting skill.

Troubleshooting

The skill does not appear in `/skill`. Discovery only reads immediate subdirectories containing a file named exactly `SKILL.md`, case-sensitive. Check for `skill.md`, for a `SKILL.md` one level too deep, and for a directory in the wrong scope. Then check the session log for `Failed to parse ... :` - a missing closing `---`, a name with uppercase characters, or malformed YAML all land there.

The skill appears but `/name` does nothing. `user-invocable: false` hides it from the menu and the trigger. The `access` column says `agent-only`.

Edits to `SKILL.md` are not picked up. The body is cached after the first invocation. Run `/reload skills`.

`$ARGUMENTS` is still visible in the text the agent received. The placeholder was written with different capitalisation - substitution is exact and case-sensitive - or the skill ran via a path that does not substitute. Check `argument-hint` and the body spelling.

A `` !`command` `` block shows the placeholder instead of output. The skill has not set `allow-dynamic-context: true`.

A `` !`command` `` block shows an error message instead of output. Either the executable is not on the allowlist or the command contains a shell metacharacter. Both refusals put an explanatory sentence in the output, which is also what the agent sees.

A tool the skill needs is refused. Either the tool is not in `allowed-tools` (add it, using the registry spelling), or it is and the entry is stale - an unresolved entry is reported once, naming the skill and the

entry, in the session log.